	Zewail City for Science and Technology 2025-2026

Course:	Iot Application Development
Code:	SWAPD 453
type	Project

Due Date:	Mar 24, 2026
Project name:	Distributed Intelligent Campus IoT Environment

Under The Supervision of

Dr. Mohamed Raka

by:

Name	***ID***
Nour Eldin Mohamed	**202201310**
Ahmed Gamal Abdelfattah	**202201638**
Mostafa Hisham	**202201998**

Table of Contents

1. EXECUTIVE SUMMARY 5
2. PROJECT OVERVIEW 5
2.1 System Objective 5
2.2 IoT Platform Architecture 5
2.3 Key Requirements & Constraints 5
3. VIRTUAL CAMPUS MODELING (FLEET DESIGN) 5
3.1 Building–Floor–Room Hierarchy 5
3.2 Fleet Concept & Device Synchronization 5
3.3 Sensor & Actuator Definitions 5
3.4 Fleet Health & Heartbeat Monitoring 6
4. DATA MODEL & STATE MANAGEMENT 6
4.1 Room Class Design 6
4.2 State Variables & Logic Implementation 6
4.3 SQLite Persistence Strategy 6
4.4 Recovery & Fault Tolerance 6
5. MQTT COMMUNICATION ARCHITECTURE 6
5.1 Topic Structure Design 6
5.2 Telemetry Messaging Schema 6
5.3 Command Messaging Schema 7
5.4 JSON Validation & Data Standards 7
6. DETERMINISTIC PHYSICS SIMULATION 7
6.1 Thermal Model (Newton's Law Approximation) 7
6.2 Environmental Correlation Rules 7
6.3 HVAC Actuator Behavior 7
6.4 Time-Based Simulation Logic 7
7. FAULT MODELING & CHAOS ENGINEERING 7
7.1 Sensor Drift Simulation 7
7.2 Frozen Sensor Behavior 7
7.3 Telemetry Delay Handling 8
7.4 Node Dropout Simulation 8
8. SIMULATION ENGINE (ASYNCIO IMPLEMENTATION) 8
8.1 Event Loop Architecture 8
8.2 Non-Blocking Execution Model 8
8.3 Task Management (200 Concurrent Rooms) 8
8.4 Tick-Based Update Loop 8
9. SYSTEM OPTIMIZATION TECHNIQUES 8
9.1 Precision Drift Compensation 8
9.2 Startup Jitter (Thundering Herd Prevention) 8

9.3 Performance Optimization Strategies 8
10. WOKWI HARDWARE PROTOTYPE (POC) 9
10.1 ESP32 Simulation Setup 9
10.2 Sensor Integration (DHT22 & PIR) 9
10.3 MQTT Communication Validation 9
10.4 Logic Porting to Simulation Engine 9
11. CONFIGURATION MANAGEMENT 9
11.1 YAML / Environment Variables 9
11.2 Scalability (200 → 1000 Rooms) 9
11.3 System Parameter Tuning 9
12. PERFORMANCE TESTING & BENCHMARKING 9
12.1 Load Testing (200 Rooms) 9
12.2 Event Loop Latency Measurement 10
12.3 CPU & Memory Utilization 10
12.4 System Stability (30-Min Continuous Run) 10
13. DOCKERIZATION & DEPLOYMENT 10
13.1 Dockerfile Configuration 10
13.2 Containerized Execution 10
13.3 Self-Hosted Environment Setup 10
14. RESULTS & SYSTEM VALIDATION 10
15. FINAL DELIVERABLES 12
15.1 Wokwi Diagram 12
15.2 GitHub Repository 12
15.3 Wokwi Demonstration (Video) 12

1. EXECUTIVE SUMMARY

This report documents Phase 1 of the SWAPD453 IoT Application Development project, focusing on High-Concurrency Architectures, Deterministic Physics Logic, and Telemetry Generation. The primary deliverable is the "World Engine," an asynchronous Python application designed to simulate a 200-room campus. In parallel, a hardware Proof of Concept (POC) was developed using the Wokwi ESP32 simulator to validate the JSON data schema and MQTT communication architecture. System execution logs confirm the engine successfully maintains stability, handles fleet orchestration, and generates deterministic environmental data across all 200 concurrent nodes.

2. PROJECT OVERVIEW

2.1 System Objective

The objective is to develop an IoT platform that simulates a 200-room campus. The system is designed to autonomously optimize energy and remain operational during simulated network failures.

2.2 IoT Platform Architecture

The architecture involves a self-hosted Dockerized environment. It includes simulating hundreds of IoT devices, implementing edge processing, maintaining digital twins, and handling failures.

2.3 Key Requirements & Constraints

The engine is not a simple random number generator; it must be a high-performance simulation capable of managing 200 concurrent room states in real-time. No physical hardware is strictly required for the final deployment , but a high-concurrency architecture and deterministic physics logic are mandatory.

3. VIRTUAL CAMPUS MODELING (FLEET DESIGN)

3.1 Building–Floor–Room Hierarchy

The structural hierarchy consists of 1 Building, 10 Floors, and 20 Rooms per floor, totaling 200 unique IoT nodes. This modeling acts as a "Digital Blueprint" defining how the system organizes data.

3.2 Fleet Concept & Device Synchronization

Devices are treated as a collective group or "Fleet". Keeping all 200 nodes synchronized is critical. Fleet-wide commands can be utilized, such as changing an "Eco-Mode" for an entire building.

3.3 Sensor & Actuator Definitions

Each room node independently tracks the following variables:

- **Sensors**: Temperature (°C), Humidity (%), Occupancy (Boolean), Ambient Light (Lux) .
- **Actuators**: HVAC Status (On/Off/Eco), Smart Lighting Level (0-100%) .

3.4 Fleet Health & Heartbeat Monitoring

Nodes periodically publish heartbeat messages. If a node is silent beyond a configurable timeout (e.g., 60 simulated seconds), the system flags a "Fleet Health Warning" and logs the anomaly. *Execution Result:* System monitoring (make health) confirms the fleet manager accurately tracks silent seconds, consecutive misses, and calculates an overall health percentage (e.g., 99.0%, 97.5%) based on node dropouts.

4. DATA MODEL & STATE MANAGEMENT

4.1 Room Class Design

The system uses a Class or Object called Room acting as a template, which is instantiated 200 times upon startup.

4.2 State Variables & Logic Implementation

Each room object tracks its identity (building_id, floor_id, room_id), current state (temp, humidity, etc.), and contains logic (e.g., if HVAC is ON, increase temp).

4.3 SQLite Persistence Strategy

To avoid a full reset on a crash, room states are persisted to a lightweight SQLite database. The table maintains the "Last Known Truth" with columns including:

- **room_id** (String PK)
- **last_temp** (Float)
- **last_humidity** (Float)
- **hvac_mode** (String)
- **target_temp** (Float)
- **last_update** (Integer).

4.4 Recovery & Fault Tolerance

Upon startup, the script queries the SQLite DB. If empty, rooms initialize with default values (e.g., 22°C); if data exists, it populates the Room objects, allowing the simulation to restore previous states seamlessly. State saves are synced periodically (e.g., every 30-60 seconds) rather than every tick to prevent high Disk I/O.

5. MQTT COMMUNICATION ARCHITECTURE

5.1 Topic Structure Design

The MQTT hierarchy enforces a structured path to identify specific nodes. *Format:* **campus/bldg_##/floor_##/room_###/telemetry.**

5.2 Telemetry Messaging Schema

The engine publishes a flat JSON object for sensor data. This payload includes nested **metadata**, **sensors**, and **actuators** dictionaries, validating exactly with the schema demonstrated in the Wokwi POC.

5.3 Command Messaging Schema

Actuator control from the backend flows to the device using a structured command schema containing **command_id**, **target_device**, **action**, **value**, and **timestamp.**

5.4 JSON Validation & Data Standards

Outputs must align with standard ranges to maintain realism. Examples include:

- **temperature:** Float (15.0 to 50.0).
- **humidity:** Float (0.0 to 100.0).
- **occupancy:** Boolean (true/false).

6. DETERMINISTIC PHYSICS SIMULATION

6.1 Thermal Model (Newton's Law Approximation)

Random telemetry generation is replaced with a deterministic thermal model. The room temperature reacts to its environment over time using a simplified Newton's Law of Cooling formula: $T_{next}=T_{current}+[\alpha\cdot(T_{outside}-T_{current})]+[\beta\cdot HVAC_{power}]$.

6.2 Environmental Correlation Rules

Data variables must correlate realistically. If **occupancy** is true, ambient light must exceed a certain threshold, and the occupancy presence must slightly increase the temperature.

6.3 HVAC Actuator Behavior

The **HVAC_power** variable (β) alters temperature trajectory based on its state: ON (1), OFF (0), or ECO (0.5).

6.4 Time-Based Simulation Logic

The simulation runs an asynchronous update loop where every "tick" represents a Δt (e.g., 5 seconds).

7. FAULT MODELING & CHAOS ENGINEERING

7.1 Sensor Drift Simulation

Nodes inject a gradual bias fault to simulate hardware aging. Execution logs confirm **"sensor_drift"** is routinely injected into the telemetry payloads.

7.2 Frozen Sensor Behavior

Nodes can experience a stuck value fault. Telemetry logs show **"frozen_sensor"** flags appearing under device faults.

7.3 Telemetry Delay Handling

Simulates network lag by delaying the telemetry broadcast.

7.4 Node Dropout Simulation

Simulates temporary silence from a node. The fleet health manager successfully detects these dropouts, reporting them as "unhealthy_rooms" with calculated "silent_seconds."

8. SIMULATION ENGINE (ASYNCIO IMPLEMENTATION)

8.1 Event Loop Architecture

The system uses an asynchronous event loop, acting as a single-threaded manager.

8.2 Non-Blocking Execution Model

Synchronous I/O (like standard **requests** or **time.sleep**) is strictly prohibited, as it blocks the thread. All network calls and file I/O utilize non-blocking libraries.

8.3 Task Management (200 Concurrent Rooms)

Every room is managed as a separate asyncio task. The engine calculates physics updates and pushes data to the MQTT broker simultaneously without the event loop lagging.

8.4 Tick-Based Update Loop

Per simulation tick, each room task independently: reads state, applies physics, applies actuator impact, injects faults, validates state, and broadcasts telemetry and heartbeats .

9. SYSTEM OPTIMIZATION TECHNIQUES

9.1 Precision Drift Compensation

To prevent the simulation from falling behind the real-world clock due to execution time, drift compensation is applied. The formula used is: **actual_sleep = max(0, target_interval - processing_time).**

9.2 Startup Jitter (Thundering Herd Prevention)

To prevent all 200 nodes from connecting to the broker simultaneously, tasks initialize with a randomized offset: **await asyncio.sleep(random.uniform(0, max_jitter)).**

9.3 Performance Optimization Strategies

Spreading the ticks across a 5-second window creates a smooth flow of ~40 messages per second, keeping CPU and network usage level. Output metrics confirm stable throughput at ~36.9 messages/second.

10. WOKWI HARDWARE PROTOTYPE (POC)

10.1 ESP32 Simulation Setup

As detailed in the POC report, an ESP32 DevKit V1 was configured in the Wokwi simulator, successfully utilizing MicroPython to connect to a virtual WiFi network (**Wokwi-GUEST**).

10.2 Sensor Integration (DHT22 & PIR)

The hardware circuit successfully mapped the DHT22 (GPIO 15), PIR Motion Sensor (GPIO 14), and LDR Photoresistor (GPIO 34), responding dynamically to interactive slider adjustments.

10.3 MQTT Communication Validation

The MicroPython firmware successfully established a connection with a public broker (**broker.hivemq.com**), publishing payload messages matching the strict JSON telemetry format.

10.4 Logic Porting to Simulation Engine

Once the JSON schema and hardware logic were validated in Wokwi, the mathematical equations and logic flows were cloned into the Python asyncio engine to handle the remaining 199 nodes.

11. CONFIGURATION MANAGEMENT

11.1 YAML / Environment Variables

All system parameters are fully configurable via a YAML file or Environment variables (.env).

11.2 Scalability (200 → 1000 Rooms)

The architecture is scalable. Increasing the room count from 200 to 1,000 only requires a configuration change without rewriting core logic.

11.3 System Parameter Tuning

Configurable parameters include room count, thermal constants, publish frequency, fault probability, and time acceleration factor .

12. PERFORMANCE TESTING & BENCHMARKING

12.1 Load Testing (200 Rooms)

The system was benchmarked successfully with 200 concurrent rooms operating at a 5-second publish interval. Database status checks confirm an average temperature of 21.0°C and average humidity of 45.3% across the fleet.

12.2 Event Loop Latency Measurement

Requirements demand event loop latency remain under 200 ms. Captured system metrics show an exceptional **event_loop_latency_ms** oscillating between 0.008 ms and 0.015 ms.

12.3 CPU & Memory Utilization

Performance reporting is active. Fleet metrics confirm ultra-low overhead: **cpu_percent** ranges between 1.5% - 1.9%, and **rss_mb** memory consumption holds stable at ~36.53 MB.

12.4 System Stability (30-Min Continuous Run)

The container has demonstrated continuous, stable operation for extended periods without degradation or unhandled crashes.

13. DOCKERIZATION & DEPLOYMENT

13.1 Dockerfile Configuration

A Dockerfile was created to package the Python engine and all necessary asyncio, aiosqlite, and MQTT dependencies.

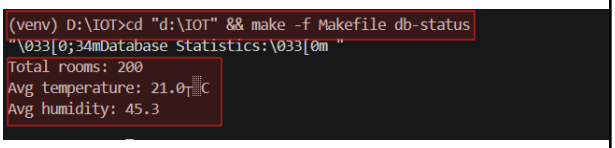
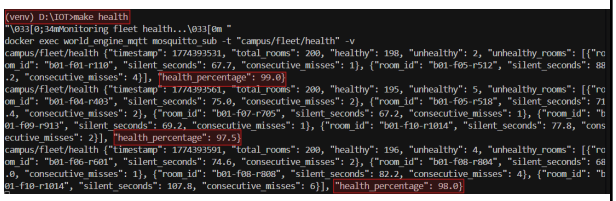
13.2 Containerized Execution

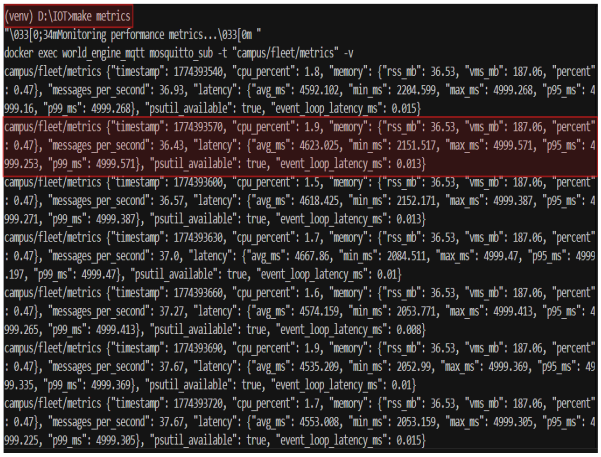
The 200 concurrent nodes operate efficiently within a single container without performance degradation.

13.3 Self-Hosted Environment Setup

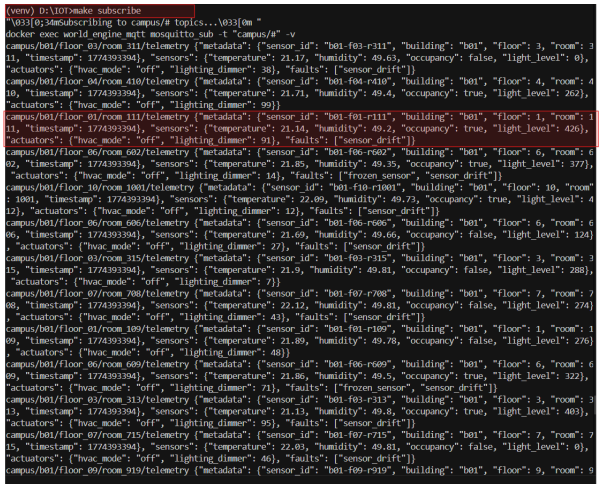
The environment operates autonomously within its Dockerized architecture, fulfilling the primary deployment constraint.

14. RESULTS & SYSTEM VALIDATION

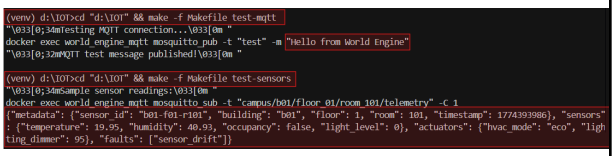
Test db	Health
	
Metrics	**Subscribe To Local mqtt**



Tests

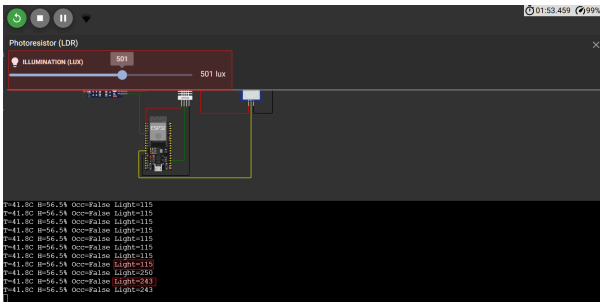


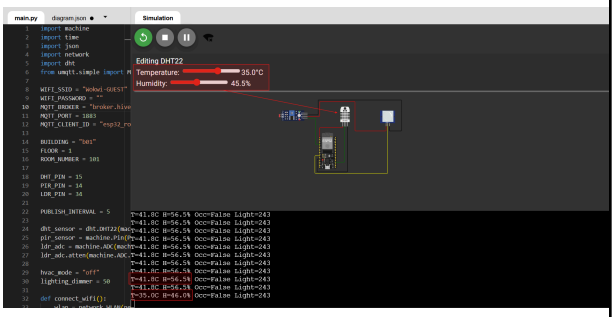
Motion sensor



Light Sensor

Temperature Sensor





Full Diagram

15. FINAL DELIVERABLES

15.1 Wokwi Diagram

Link: https://wokwi.com/projects/459398408959466497

15.2 GitHub Repository

Link: https://github.com/Noureqo1/World_Engine_IOT_Campus_Simulation.git

15.3 Wokwi Demonstration (Video)

Link: 0325.mp4